TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 3 · DEVSECOPS & CLOUD SECURITY

WEEK 10

DevOps Foundations and DevSecOps Pipeline Security

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**10 of 12**	**3 — DevSecOps & Cloud Security**	**AI-First (Manual → AI Enhancement)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 10: DevOps Foundations and DevSecOps Pipeline Security**
  - Phase: DevSecOps and Cloud Security (Weeks 9–12)
  - Day 1 — DevOps Lifecycle and Shift-Left Security
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Git Version Control and Secure Repository Practices
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — CI/CD Pipelines with GitHub Actions
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — SAST, Dependency Scanning, and Secrets Detection
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio (Mini Project 3 Kickoff)
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 11**

Phase: DevSecOps and Cloud Security (Weeks 9–12) **Week 10 of 12**

WEEK 10: DEVOPS FOUNDATIONS AND DEVSECOPS PIPELINE SECURITY

Weekly Overview

This week you transition from traditional security operations to building security directly into the software development lifecycle. You will learn the DevOps lifecycle (Plan → Code → Build → Test → Release → Operate → Monitor), the concept of 'shifting left' (finding vulnerabilities earlier when they are cheaper to fix), and the three pillars of DevSecOps. You will gain hands-on experience with Git, GitHub Actions CI/CD pipelines, and security scanning tools including SAST (Static Application Security Testing), dependency scanning, and secrets detection. By the end of this week, you will have configured a complete CI/CD pipeline with automated security gates that block vulnerable code from reaching production.

How this week builds on Week 9

- Week 9 taught you to communicate findings in professional reports. Now you will automate those findings — instead of manually reporting a vulnerability, your pipeline will automatically reject code that contains it.

How this week sets up Week 11

- Week 11 covers container security and Infrastructure as Code (IaC). The CI/CD pipeline you build this week will scan containers and infrastructure templates for vulnerabilities.

Why this matters in a real security job

- Traditional security finds vulnerabilities in production (expensive to fix). DevSecOps finds them in development (cheap to fix). The difference is 100x cost savings and preventing breaches before they happen.
- Organisations without DevSecOps are 3x more likely to have a breach.

■ **DevSecOps Week Note: Scripts and pipeline files ARE the lesson this week. Every line of YAML, every security gate, and every scan command must be explained in plain English. Your deliverable is a working pipeline, not just a report.**

DAY 1

DevOps Lifecycle and Shift-Left Security

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the DevOps infinite loop (Plan → Code → Build → Test → Release → Operate → Monitor)

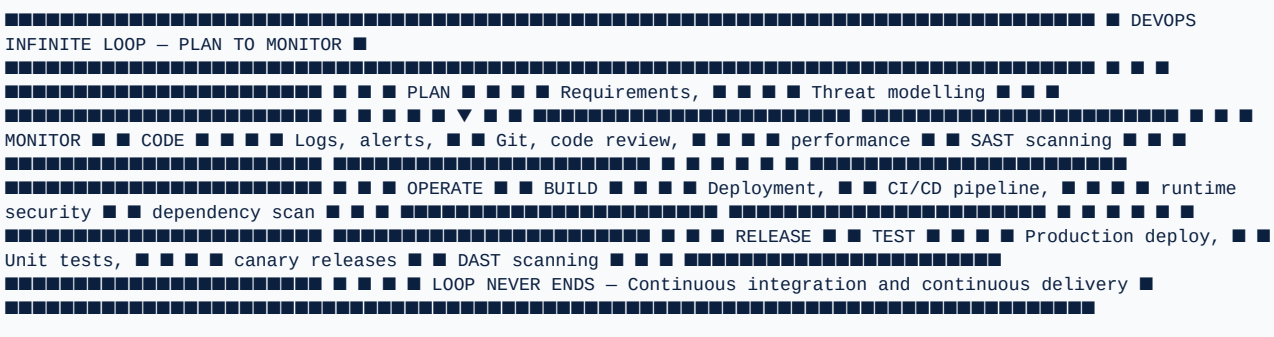
2. Define 'shift-left security' and explain why finding vulnerabilities earlier reduces cost

3. Compare traditional security with DevSecOps

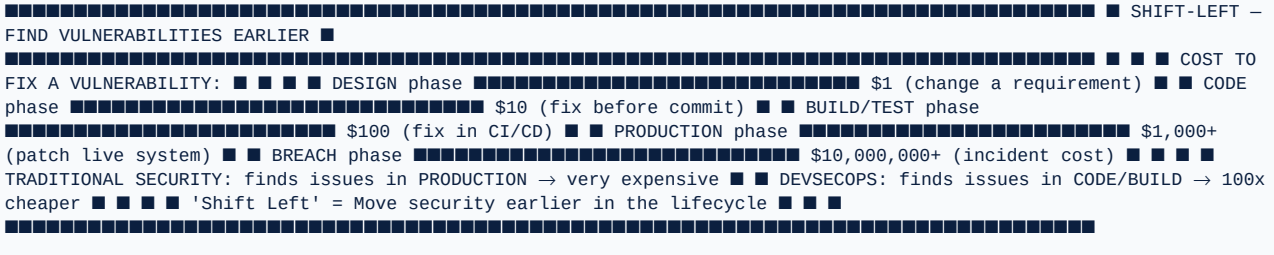
DEVOPS

is a set of practices that combines software development (Dev) and IT operations (Ops) to shorten the development lifecycle while delivering features, fixes, and updates frequently. DevSecOps integrates security into every phase of the DevOps lifecycle.

DevOps Infinite Loop — Plan to Monitor



Shift-Left — Find Vulnerabilities Earlier



Traditional Security vs DevSecOps

Aspect	Traditional Security	DevSecOps
When security runs	After development (pre-release)	Throughout every phase

Who does security	Security team only	Everyone (Dev + Ops + Sec)
How vulnerabilities found	Manual pen test, code review	Automated SAST, dependency scan
Time to fix	Weeks (release held)	Minutes (pipeline blocks commit)
Cost to fix	High (requires code changes)	Low (fix at the source)
Developer feedback	Delayed (end of sprint)	Immediate (on every commit)
Security culture	Separate silo	Shared responsibility

Three Pillars of DevSecOps

- People: security training for developers, shared responsibility, security champions in each team
- Process: security gates in CI/CD pipeline, threat modelling in planning, security requirements alongside functional requirements
- Technology: SAST, DAST, dependency scanning, secrets detection, container scanning, IaC scanning — all automated

WHY THIS MATTERS

The 2017 Equifax breach (148 million records) was caused by an unpatched Apache Struts library (CVE-2017-5638). The patch had been available for 2 months. A basic dependency scanner in a CI/CD pipeline would have flagged the vulnerable library on every build. The breach cost $575 million in settlements. One automated security gate would have prevented it.

PRACTICE *(2 Hours)*

LAB 10.1.1 — DevSecOps Maturity Assessment

Objective: Assess a fictional organisation's DevSecOps maturity.

Prerequisites: None — scenario-based

QuickPayNG (the Nigerian fintech from earlier weeks) uses the following development practices:

- Code: developers push directly to main branch
- Build: Jenkins CI server runs unit tests only
- Test: manual QA team tests before each monthly release
- Deploy: IT team manually deploys to production
- Security: annual penetration test; no automated scanning

For each DevOps phase (Plan, Code, Build, Test, Release, Operate, Monitor), assess QuickPayNG's current security maturity (None/Low/Medium/High) and recommend one specific improvement.

■ ANALYSE THIS

QuickPayNG pushes directly to main with no review. How many vulnerabilities could be introduced per month if 10 developers each commit 5 times daily? What is the first security gate you would add and why?

LAB 10.1.2 — Shift-Left Cost Calculation

Objective: Calculate the cost savings from shifting security left.

Prerequisites: None — scenario-based

QuickPayNG discovers a SQL injection vulnerability. Calculate the cost of fixing it at each phase, using the multipliers from the Shift-Left diagram. Then calculate: if QuickPayNG ships 50 features per month and 10% have security vulnerabilities — what is the annual cost difference between finding them in CODE vs PRODUCTION?

■ ANALYSE THIS

If implementing SAST costs $10,000/year and saves $500,000 in avoided breach costs — what is the ROI? Present this as a business case for QuickPayNG's CTO.

BUILD *(2 Hours)*

BUILD BRIEF

Create a DevSecOps Roadmap for QuickPayNG — a 90-day plan to transform their development security posture.

Phase 1 — Week 1–2 (Quick Wins):

- Implement branch protection on main (no direct pushes — require pull requests)
- Add .gitignore to prevent accidental secret commits
- Integrate a free SAST tool (Bandit for Python, Semgrep for any language)

Phase 2 — Week 3–6 (Pipeline Security):

- Set up GitHub Actions CI pipeline that runs SAST on every pull request
- Add dependency scanning (pip-audit for Python, npm audit for Node.js)
- Configure pipeline to FAIL if High/Critical vulnerabilities found

Phase 3 — Week 7–12 (Mature Programme):

- Add secrets detection (GitLeaks or GitHub Secret Scanning)
- Implement DAST scanning on staging environment
- Security training for all developers (OWASP Top 10 awareness)

For each item:

- Tool name, setup time, cost, OWASP/CVE category it prevents

Format:

- Professional project plan with timeline, responsibilities, and success metrics

DOCUMENT

Writing Prompt:

Write a 'Security Champions Programme' proposal for QuickPayNG. A Security Champion is a developer who takes extra security responsibility within their team. Audience: CTO. Length: 400–500 words.

- Include: What a Security Champion does (code reviews, security training, pipeline maintenance), time commitment (2 hours/week), how to select champions, incentives (recognition, training budget), expected outcomes (reduced vulnerabilities, faster fixes)

DAY
2

Git Version Control and Secure Repository Practices

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Use essential Git commands (init, clone, add, commit, push, pull, branch, merge)

2. Configure branch protection rules to enforce security reviews

3. Identify and prevent common Git security mistakes (secrets in commits, force pushes)

GIT

is a distributed version control system that tracks changes to source code over time, enabling multiple developers to collaborate, maintain history, and recover from mistakes.

Git Workflow — Feature Branch Model



Essential Git Commands — Security-Relevant

Command	Purpose	Security Relevance
git init	Create new repository	Start tracking code changes
git clone [url]	Copy remote repository locally	Get existing codebase
git add [file]	Stage changes for commit	Review what you're committing — don't add secrets!
git commit -m 'msg'	Save staged changes with message	Audit trail — messages should be meaningful
git push origin [branch]	Upload commits to remote	Code becomes visible to team
git pull	Download and merge remote changes	Keep local up-to-date
git branch [name]	Create new branch	Isolate feature work from main

git checkout [branch]	Switch to branch	Move between branches
git log --oneline	View commit history	Audit trail of all changes
git diff	Show uncommitted changes	Review before committing
git reset HEAD~1	Undo last commit (keep changes)	Remove accidental commits
git rm --cached [file]	Untrack file (keep locally)	Remove accidentally committed secrets

Common Git Security Mistakes

Mistake	Example	Prevention
Committing secrets	git commit -m 'fix' with DB_PASSWORD=secret123 in .env	Use .gitignore; use secrets management (environment variables)
Force pushing to main	git push --force origin main (rewrites history)	Enable branch protection — block force pushes
No code review	Direct commits to main without PR	Require pull request and review before merge
Merge without CI passing	Override failed security checks	Require status checks to pass before merge
Public repository with secrets	Open-source repo with API keys in commit history	Rotate keys immediately; use git-filter-repo to remove from history
No .gitignore	Committing .env, *.key, node_modules	Create .gitignore before first commit

GitHub Branch Protection Rules

- Require pull request reviews: minimum 1 reviewer must approve before merging
- Require status checks to pass: CI pipeline (SAST, tests) must pass before merge
- Require signed commits: cryptographic proof of author identity (GPG signing)
- Block force pushes: prevent history rewriting (cannot delete evidence of a commit)
- Require up-to-date branches: must be current with main before merging
- Restrict who can push: only specific teams can push to protected branches

WHY THIS MATTERS

In 2022, a developer at a major Nigerian bank accidentally committed AWS access keys to a public GitHub repository. The keys were discovered by automated bots within 4 minutes. By the time the developer noticed and tried to delete the commit, attackers had already provisioned cloud resources. The attack cost $23,000 in cloud bills before the keys were revoked. GitLeaks running in a pre-commit hook would have prevented the commit.

PRACTICE *(2 Hours)*

LAB 10.2.1 — Set Up a Secure Git Repository

Objective: Configure a GitHub repository with branch protection and security settings.

Prerequisites: GitHub account; Git installed on Kali

Step 1 — Create a GitHub Repository

Go to github.com → New Repository → Name: techrise-devsecops → Private → Add README. Clone to your local machine:

BASH

```
git clone https://github.com/YOUR-USERNAME/techrise-devsecops cd techrise-devsecops
```

Step 2 — Configure Git Identity

BASH

```
git config --global user.name 'Your Name' git config --global user.email 'your@email.com'
```

Step 3 — Create a .gitignore File

BASH

```
cat > .gitignore << 'EOF' # Environment files (NEVER commit these) *.env .env* *.env.local # Secret files *.key *.pem *.p12 *.pfx # Dependencies node_modules/ venv/ __pycache__/ # IDE files .vscode/ .idea/ EOF git add .gitignore git commit -m 'Add .gitignore to prevent secret commits' git push origin main
```

Step 4 — Create a Feature Branch and Simulate Development

BASH

```
git checkout -b feature/payment-api # Create a simple Python file with a vulnerability cat > payment.py << 'EOF' import sqlite3 def get_payment(user_id): conn = sqlite3.connect('payments.db') # VULNERABLE: SQL injection - user_id not sanitised query = f'SELECT * FROM payments WHERE user_id = {user_id}' return conn.execute(query).fetchall() EOF git add payment.py git commit -m 'Add payment API endpoint' git push origin feature/payment-api
```

Step 5 — Enable Branch Protection on GitHub

GitHub repository → Settings → Branches → Add branch protection rule. Branch name pattern: main. Enable: Require a pull request before merging, Require status checks to pass, Block force pushes.

■ ANALYSE THIS

The payment.py file has a SQL injection vulnerability. In a DevSecOps pipeline, at which exact stage would this be caught? (Hint: SAST in the CI pipeline). What would happen to the developer's pull request if SAST finds this vulnerability?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Git Security Policy document for QuickPayNG's development team.

Section 1 — Repository Setup Standards:

- Required .gitignore entries (list the top 10 patterns to always exclude)
- Repository visibility policy (when public vs private)

- Naming conventions for branches (feature/, hotfix/, release/)

Section 2 — Commit Standards:

- Commit message format (what makes a useful audit trail)
- Pre-commit checklist (5 things to verify before every commit)
- What to do if you accidentally commit a secret

Section 3 — Branch Protection Requirements:

- List all protection rules to enable on main/production branches
- Code review requirements (minimum reviewers, who can approve)

Section 4 — Incident Response for Git Secrets:

- Step-by-step: what to do when a secret is committed to a public repo
- Step-by-step: how to remove a secret from git history

DOCUMENT

Writing Prompt:

Write a Git Training Module introduction for QuickPayNG's junior developers who have never used version control. Audience: recent graduates joining the team. Length: 300–400 words.

- Include: Why version control exists (collaboration, history, recovery), why security is everyone's responsibility (not just the security team), the three things that should NEVER be committed (secrets, large files, generated code), and the one habit that will save their career (always review git diff before committing)

DAY
3

CI/CD Pipelines with GitHub Actions

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain CI/CD concepts and the role of automated pipelines in DevSecOps

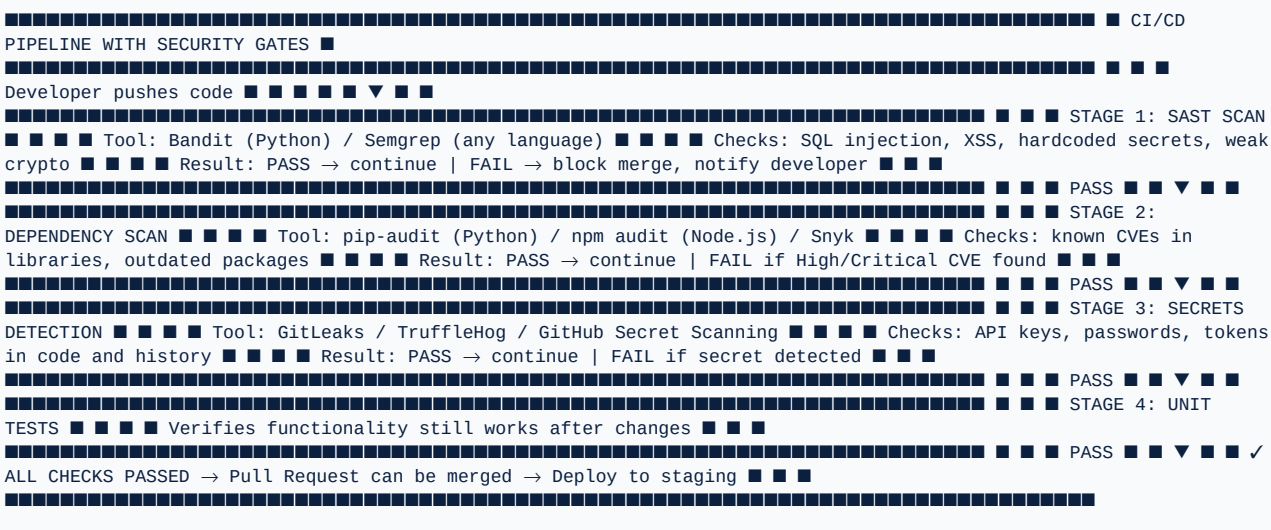
2. Write a basic GitHub Actions workflow in YAML

3. Add security gates (SAST, dependency scan) to a CI/CD pipeline

CI/CD

(Continuous Integration / Continuous Delivery) is the practice of automatically building, testing, and deploying code changes. Every commit triggers a pipeline that runs automated checks — security gates fail the pipeline if vulnerabilities are found.

CI/CD Pipeline with Security Gates



GitHub Actions YAML — Structure

YAML

```
# .github/workflows/security.yml name: DevSecOps Pipeline # TRIGGER: when does this pipeline run? on: push: branches: [ main, 'feature/**' ] pull_request: branches: [ main ] # JOBS: what work to do jobs: security-scan: runs-on: ubuntu-latest # VM environment steps: # Step 1: Get the code - name: Checkout code uses: actions/checkout@v4 # Step 2: Set up Python - name: Set up Python uses: actions/setup-python@v4 with: python-version: '3.11' # Step 3: Install tools - name: Install security tools run: | pip install bandit pip-audit # Step 4: SAST scan - name: Run SAST (Bandit) run: | bandit -r . -f json -o bandit-results.json || true bandit -r . --severity-level high # Step 5: Dependency scan - name: Check dependencies run: | pip-audit --requirement requirements.txt
```

Reading GitHub Actions Output

- Green checkmark (✓): all steps passed — code is safe to merge
- Red X: one or more steps failed — pipeline blocked, developer must fix
- Yellow circle: pipeline is running — wait for results
- View logs: click on the failed step to see exactly what went wrong
- Annotations: GitHub shows security findings directly in the pull request diff

WHY THIS MATTERS

In 2020, GitHub reported that 3 million repositories had exposed secrets in commits. Within minutes of being committed, bots automatically scan GitHub for API keys, AWS credentials, and passwords. GitHub Secret Scanning (free for public repos) and tools like TruffleHog detect secrets before they can be exploited. A 5-line YAML addition to your pipeline prevents multi-million dollar breaches.

PRACTICE *(2 Hours)*

LAB 10.3.1 — Build Your First Security Pipeline

Objective: Create a GitHub Actions workflow with SAST and dependency scanning.

Prerequisites: GitHub repository from Day 2; Python installed

Step 1 — Create the Workflow Directory

BASH

```
mkdir -p .github/workflows nano .github/workflows/security.yml
```

Step 2 — Write the Basic Pipeline

Add this YAML content — explained line by line:

security.yml — Basic DevSecOps Pipeline

```
name: TechRise DevSecOps Pipeline on: push: branches: [ main, 'feature/**' ] pull_request: branches: [ main ] jobs: security: runs-on: ubuntu-latest steps: - name: Checkout code uses: actions/checkout@v4 with: fetch-depth: 0 # Full history for secret scanning - name: Set up Python 3.11 uses: actions/setup-python@v4 with: python-version: '3.11' - name: Install security tools run: pip install bandit pip-audit - name: SAST - Run Bandit run: bandit -r . --severity-level high --exit-zero - name: Dependency Scan - pip-audit run: pip-audit || true
```

Step 3 — Create a Requirements File

BASH

```
cat > requirements.txt << 'EOF' flask==2.0.1 requests==2.25.1 cryptography==36.0.0 EOF
```

Step 4 — Commit and Push

BASH

```
git add .github/ requirements.txt git commit -m 'Add DevSecOps security pipeline' git push origin feature/payment-api
```

Go to your GitHub repository → Actions tab. Watch your pipeline run. Click on each step to see the output.

Step 5 — Create a Pull Request

GitHub → Pull Requests → New pull request → feature/payment-api → main. The pipeline will run automatically. Check: does Bandit find the SQL injection in payment.py? Does pip-audit find any vulnerable dependencies?

■ ANALYSE THIS

Your Bandit SAST scan should have flagged the SQL injection in payment.py (B608: possible SQL injection via string-based query construction). What is the severity level Bandit assigns? What is the remediation Bandit suggests? How would you fix payment.py to make the scan pass?

BUILD *(2 Hours)*

BUILD BRIEF

Enhance your security pipeline with additional security gates.

Add these steps to your security.yml:

- Secrets detection with GitLeaks: uses: gitleaks/gitleaks-action@v2
- Semgrep SAST (broader language support): uses: returntocorp/semgrep-action@v1
- Safety check (Python-specific CVE database): pip install safety && safety check
- OWASP dependency check report generation

Pipeline quality requirements:

- Pipeline must FAIL (not warn) on High/Critical severity findings
- Each step must have a clear name explaining what it checks
- Add a notification step: on failure, output a summary of what failed

Documentation requirement:

- For each tool added: what it checks, what OWASP category it addresses, sample output

Deliverable:

- Working .github/workflows/security.yml file that runs all 5+ security checks

DOCUMENT

Writing Prompt:

Write a CI/CD Security Gate Policy for QuickPayNG. This defines what the automated pipeline must check and what constitutes a pass or fail. Audience: Development Manager.

- Required: List of mandatory security checks (minimum 4), failure criteria (what severity triggers a block), override process (who can approve overrides and under what circumstances), SLA for fixing failed security checks (High: 24h, Critical: 4h)
- This policy document is what gives your pipeline legal authority to block deployments

DAY 4

SAST, Dependency Scanning, and Secrets Detection

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Distinguish SAST, DAST, and SCA and explain when each is used

2. Run Bandit and Semgrep manually to find vulnerabilities in code

3. Configure dependency scanning to detect known CVEs in libraries

SAST (STATIC APPLICATION SECURITY TESTING)

analyses source code without executing it to find security vulnerabilities — SQL injection, XSS, hardcoded credentials, insecure functions — before the code runs.

SAST vs DAST vs SCA

Type	Full Name	What It Tests	When	Example Tools
SAST	Static Application Security Testing	Source code (without running it)	At commit / in CI	Bandit, Semgrep, SonarQube, Checkmarx
DAST	Dynamic Application Security Testing	Running application (black box)	After deploy to staging	OWASP ZAP, Burp Suite Pro, Nikto
SCA	Software Composition Analysis	Third-party libraries and dependencies	At build time	pip-audit, npm audit, Snyk, OWASP Dependency-Check
Secrets	Secrets Detection	Hard-coded credentials in code/history	At commit (pre-commit hook)	GitLeaks, TruffleHog, GitHub Secret Scanning

Bandit — Python SAST Tool

BASH

```
# Install pip install bandit --break-system-packages # Scan a single file bandit payment.py # Scan entire project bandit -r . # Scan with severity threshold (only HIGH and CRITICAL) bandit -r . --severity-level high # Output as JSON (for CI integration) bandit -r . -f json -o bandit-results.json # Example output: # Test ID: B608 # Issue: [B608:hardcoded_sql_expressions] Possible SQL injection via string-based query # Severity: MEDIUM # Confidence:MEDIUM # Location: payment.py:6:13
```

Semgrep — Multi-Language SAST

BASH

```
# Install pip install semgrep --break-system-packages # Scan with OWASP Top 10 rules semgrep
--config=p/owasp-top-ten . # Scan for specific vulnerability class semgrep --config=p/python-security
. # Scan with auto-selection of best rules semgrep --config=auto . # Scan and output results semgrep
--config=p/security-audit . --json | python3 -m json.tool
```

Dependency Scanning — SCA Tools

BASH

```
# pip-audit — Python dependency scanner pip install pip-audit --break-system-packages pip-audit
pip-audit --requirement requirements.txt # Example output: # Name Version ID Fix Versions # flask
2.0.1 PYSEC-2023-62 2.3.2 # requests 2.25.1 CVE-2023-32681 2.31.0 # npm audit — Node.js dependency
scanner npm audit npm audit --audit-level=high # Only fail on High+ # Safety — Python CVE database pip
install safety && safety check
```

Secrets Detection — GitLeaks and TruffleHog

BASH

```
# GitLeaks — scan current repository docker run -v "$(pwd):/path" zricethezav/gitleaks:latest detect
--source /path # TruffleHog — scan git history pip install trufflehog --break-system-packages
trufflehog git file://. --only-verified # GitHub CLI — enable secret scanning gh secret set MY_SECRET
# Store secrets properly (never in code) # Example detected secret: # ■ Found verified result! #
Detector Type: AWS # Decoder Type: PLAIN # Raw result: AKIA... # Commit: a3b4c5... Author:
dev@company.com
```

WHY THIS MATTERS

In 2023, a developer at a Fortune 500 company committed AWS credentials to a private GitHub repository. Within hours, attackers (who had also compromised the GitHub account) found the credentials and exfiltrated 2TB of customer data. The breach cost $50 million. GitLeaks running as a pre-commit hook would have prevented the commit entirely — in under 1 second.

PRACTICE *(2 Hours)*

LAB 10.4.1 — Hands-On SAST with Bandit and Semgrep

Objective: Find and fix real vulnerabilities in code using SAST tools.

Prerequisites: Python; Bandit; Semgrep installed; payment.py from Day 2

Step 1 — Create a Vulnerable Python File

BASH

```
cat > vulnerable_app.py << 'EOF' import sqlite3 import hashlib import subprocess import os #
Vulnerability 1: SQL Injection def get_user(username): conn = sqlite3.connect('users.db') query =
'SELECT * FROM users WHERE name = ' + username return conn.execute(query).fetchall() # Vulnerability
2: Weak cryptography (MD5) def hash_password(password): return
hashlib.md5(password.encode()).hexdigest() # Vulnerability 3: Command injection def
list_files(directory): return os.system('ls ' + directory) # Vulnerability 4: Hardcoded secret
AWS_SECRET = 'AKIAIOSFODNN7EXAMPLE' DB_PASSWORD = 'admin123' EOF
```

Step 2 — Run Bandit

BASH

```
bandit vulnerable_app.py # Expected findings: # B608: SQL injection (MEDIUM) # B324: Use of weak MD5 hash function (HIGH) # B602: Command injection via os.system (HIGH) # B105: Hardcoded password (LOW)
```

Step 3 — Run Semgrep

BASH

```
semgrep --config=p/python-security vulnerable_app.py # Semgrep will find same + additional issues # Compare findings between Bandit and Semgrep — what did each miss?
```

Step 4 — Fix the Vulnerabilities

BASH

```
cat > fixed_app.py << 'EOF' import sqlite3 import hashlib import os # Fix 1: Parameterised query (prevents SQL injection) def get_user(username): conn = sqlite3.connect('users.db') query = 'SELECT * FROM users WHERE name = ?' return conn.execute(query, (username,)).fetchall() # Fix 2: Use bcrypt/SHA256 (not MD5) def hash_password(password): return hashlib.sha256(password.encode()).hexdigest() # Fix 3: Use subprocess.run with list (no shell injection) def list_files(directory): return subprocess.run(['ls', directory], capture_output=True) # Fix 4: Use environment variables (not hardcoded) AWS_SECRET = os.environ.get('AWS_SECRET_KEY') EOF # Run Bandit on fixed file — should have no HIGH findings bandit fixed_app.py
```

Step 5 — Run Dependency Scanner

BASH

```
pip-audit --requirement requirements.txt # Record: which packages have CVEs? What is the fix version? # Update requirements.txt with patched versions
```

■ ANALYSE THIS

Bandit found the MD5 hash as HIGH severity but the hardcoded password as LOW. Why is a hardcoded AWS credential more dangerous than weak password hashing in most contexts? When would LOW severity findings become critical? (Hint: business context from Week 9)

BUILD *(2 Hours)*

BUILD BRIEF

Create a Security Scanning Integration Report for your CI/CD pipeline.

Section 1 — Tools Configured:

- For each tool: name, version, what it checks, OWASP categories covered, configuration options used

Section 2 — Findings from vulnerable_app.py:

- Complete table: Tool, Finding, Severity, OWASP Category, Line Number, Remediation

Section 3 — Fixed Code Comparison:

- Side-by-side: vulnerable code vs fixed code for each finding, with explanation of why the fix works

Section 4 — Pipeline Integration:

- Which stage each tool runs in (pre-commit, CI, post-deploy), estimated runtime per scan, impact on developer workflow

Section 5 — False Positive Analysis:

- Did any tool flag something that isn't actually a vulnerability? How would you tune it?

Format:

- Professional report — professional enough to show a development team why they need these tools

DOCUMENT

Writing Prompt:

Write a Secure Coding Standards document for QuickPayNG's Python development team. Audience: Developers (technical). Length: 400–500 words.

- Required: Top 5 coding rules (with good vs bad code examples), how to use SAST locally before committing (developer workflow), what to do when the pipeline fails (fix process, not bypass process), references to OWASP Top 10 categories each rule addresses
- Tone: Practical and developer-friendly — not a security lecture

DAY
5

Review, Assessment & Portfolio (Mini Project 3 Kickoff)

LEARN *(1 Hour) — Week Review*

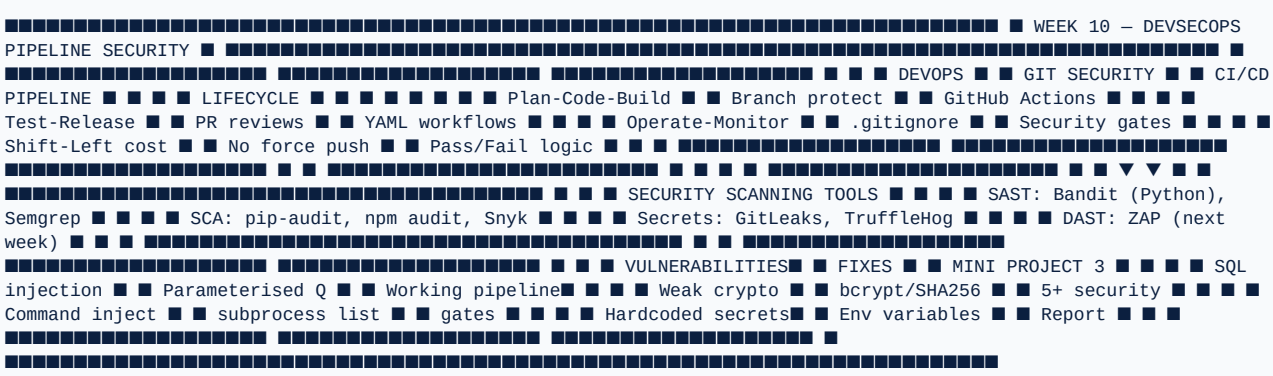
LEARNING OBJECTIVES

1. Demonstrate all Week 10 DevSecOps skills without referencing notes

2. Complete a skills assessment covering pipeline configuration

3. Kick off Mini Project 3 — Complete DevSecOps Pipeline

Week 10 Mind Map — DevSecOps Pipeline Security



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 10.5.1 — Pipeline YAML Challenge

Objective: Write a GitHub Actions pipeline from memory.

Prerequisites: No notes — 30-minute timed challenge

Write a complete GitHub Actions workflow (security.yml) from memory that:

1. Triggers on push to main and pull requests
2. Sets up Python 3.11
3. Installs and runs Bandit SAST
4. Installs and runs pip-audit dependency scanner
5. Fails the pipeline if any HIGH severity vulnerability is found

LAB 10.5.2 — Vulnerability Fix Challenge

Objective: Fix 5 vulnerabilities in code without referencing notes.

Prerequisites: Python; Bandit; no notes

Run Bandit on vulnerable_app.py. For each HIGH severity finding: write the fix, explain why it works, and re-run Bandit to confirm the finding is resolved.

LAB 10.5.3 — Git Security Audit

Objective: Audit a repository for security issues.

Prerequisites: GitHub repository from Day 2; no notes

1. Check: does the repository have branch protection enabled on main?
2. Check: is there a .gitignore that excludes .env files?
3. Check: does the GitHub Actions pipeline fail on High severity vulnerabilities?
4. Run GitLeaks or TruffleHog on the repository — are there any secrets in history?
5. Write a 1-page security audit report of your repository findings

BUILD *(2 Hours) — Mini Project 3 Kickoff*

■ MINI PROJECT 3 — Complete DevSecOps Pipeline — Secure Python Application

Build a complete DevSecOps pipeline for a fictional payment processing application. This is your major portfolio piece for Phase 3 — it demonstrates that you can implement security directly into the development lifecycle.

Deliverables:

- Repository: Create a new GitHub repository called 'techrise-payment-api' with proper branch protection enabled
- Application Code: Write a simple Python Flask API with at least one endpoint — include both a vulnerable version and a fixed version in separate branches
- Security Pipeline: GitHub Actions workflow with minimum 4 security gates (SAST, dependency scan, secrets detection, and one more of your choice)
- Vulnerability Report: run all tools against the vulnerable branch — document all findings in a table (tool, finding, severity, OWASP category, fix)
- Fixed Branch: create a 'secure' branch where all findings are remediated — pipeline passes with no High/Critical findings
- SAST Comparison: compare Bandit vs Semgrep results on the same code — what did each find? What did each miss?
- AI Integration: use AI to help write YAML pipeline configuration, then validate every line — document what AI generated vs what you corrected
- DevSecOps Maturity Report: 1-page assessment of your pipeline's coverage vs OWASP Top 10 — which categories are covered, which are not
- BONUS: Add a pre-commit hook that runs Bandit locally before any commit
- BONUS: Add OWASP ZAP DAST scan against the running application

Tools: Git · GitHub Actions · Bandit · Semgrep · pip-audit · GitLeaks · Python Flask · Claude AI (documented)

Mini Project 3 — Submission Details

- Due: End of Week 11 (one week from today)
- Format: GitHub repository link + PDF report (minimum 15 pages)
- Submission: Share repository link + PDF to instructor
- Evaluation: Working pipeline (35%), Vulnerability findings (25%), Fixed code quality (20%), Documentation (20%)

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 10 learning reflection. Length: 300–400 words.

- The most powerful security gate you added to your pipeline and why it matters
- How does automating security checks change the developer experience compared to manual code review?
- What surprised you most about the vulnerabilities Bandit or Semgrep found in code you wrote?
- How does the 'shift-left' concept from Week 10 connect to the OWASP Top 10 from Week 5?
- Prediction: which security scanning tool will be most valuable in your career and why?

Weekly Checklist (Student-Facing)

- ■ I can explain the DevOps lifecycle (all 7 phases) and where security fits in each
- ■ I can explain shift-left security and calculate the cost difference of finding vulns in code vs production
- ■ I have a GitHub repository with branch protection enabled and a .gitignore configured
- ■ I have created a GitHub Actions workflow that runs on every push and pull request
- ■ My pipeline runs Bandit SAST and reports findings
- ■ My pipeline runs pip-audit dependency scanner
- ■ I have found and fixed at least 3 vulnerabilities using SAST tools
- ■ I have run secrets detection and confirmed no secrets are in my repository history
- ■ My Mini Project 3 repository is created and the vulnerable application branch is pushed
- ■ I understand the difference between SAST, DAST, and SCA and when to use each

Skills Verification

Skill	How to Demonstrate It	Pass Standard
DevOps lifecycle	Name all 7 phases and one security activity per phase	All 7 phases correct with security context
Git workflow	Create branch, commit, push, open PR — without notes	All steps completed; PR triggers pipeline
GitHub Actions	Show working security.yml running on GitHub	Pipeline runs; at least 2 security checks visible
Bandit SAST	Run Bandit on vulnerable_app.py and show findings	SQL injection and weak crypto findings identified
Fix vulnerabilities	Show fixed_app.py with Bandit reporting 0 HIGH findings	All HIGH findings remediated; Bandit passes
Dependency scan	Run pip-audit on requirements.txt with CVE findings	At least one CVE identified with fix version
Secrets detection	Show repository history is clean of secrets	GitLeaks/TruffleHog reports no verified secrets

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Concept Questions

1. Explain 'shift-left security' in one sentence. If a vulnerability costs $10 to fix in code, what does it cost in production, and why?
2. What is the difference between SAST and DAST? Give one tool for each and describe what it would find that the other misses.
3. What is a CI/CD pipeline? In a DevSecOps pipeline, what happens when a security gate FAILS — specifically what is blocked and what must the developer do?

YAML and Pipeline Questions

4. Write a GitHub Actions step (in YAML) that runs Bandit on the current directory and fails if any HIGH severity vulnerability is found.
5. What does the 'on: pull_request: branches: [main]' trigger in GitHub Actions mean? Why is it better to run security checks on pull requests rather than after merging to main?

Tool Knowledge

6. You run pip-audit on a Flask application and get: 'flask 2.0.1 has PYSEC-2023-62, fix: 2.3.2'. What does this mean? What exactly do you do to fix it? How do you verify the fix resolved the issue?
7. A developer says: 'I already use a private GitHub repository, so I don't need secrets detection.' Why is this incorrect? Give two scenarios where secrets in a private repo are still at risk.

Code Analysis

8. Identify the vulnerabilities in this code and name the Bandit finding ID for each:

PYTHON CODE

```
import os, hashlib def authenticate(user, pwd): h = hashlib.md5(pwd.encode()).hexdigest()
os.system('log_auth ' + user) return h == DB_PASSWORD
```

Connection to Previous Weeks

9. In Week 5 you exploited SQL injection manually using Burp Suite and DVWA. In Week 10 you found SQL injection using Bandit. Compare the two approaches: which is faster, which finds more context, which can be automated, and when would you use each?

Additional Resources

DevSecOps Fundamentals

- OWASP DevSecOps Guideline: https://owasp.org/www-project-devsecops-guideline/
- NIST DevSecOps Practice Guide: https://www.nist.gov/system/files/documents/2022/06/21/NIST.SP.800-204C.pdf
- TryHackMe — DevSecOps Path: https://tryhackme.com/path/outline/devsecops

SAST Tools

- Bandit — Python SAST: https://bandit.readthedocs.io
- Semgrep — Multi-language SAST (Free): https://semgrep.dev
- SonarQube Community (Free): https://www.sonarqube.org

Dependency and Secrets Scanning

- pip-audit: https://pypi.org/project/pip-audit/
- GitLeaks: https://github.com/gitleaks/gitleaks
- TruffleHog: https://github.com/trufflesecurity/trufflehog
- Snyk (Free tier): https://snyk.io

GitHub Actions

- GitHub Actions Documentation: https://docs.github.com/en/actions
- GitHub Actions Security Guides: https://docs.github.com/en/actions/security-guides
- Starter workflows for security: https://github.com/actions/starter-workflows/tree/main/code-scanning

Looking Ahead — Week 11

Preview — What's Coming in Week 11

- Container Security — Docker and Kubernetes security best practices
- Infrastructure as Code (IaC) Security — Terraform, CloudFormation scanning
- Cloud Security Posture Management (CSPM) — AWS/Azure misconfigurations
- Adding container scanning to your CI/CD pipeline (Trivy, Grype)
- Your Week 10 pipeline becomes the foundation for scanning containers and IaC

What to Review Before Week 11

- Your GitHub Actions pipeline from Week 10 — you will add container scanning to it
- Week 4 reconnaissance — cloud misconfigurations are scanned similarly to network vulnerabilities
- Week 6 SIEM — cloud security events will flow into your SIEM

What to Set Up Before Week 11

- Install Docker on Kali: sudo apt install docker.io -y && sudo systemctl start docker
- Create a free AWS account (or use existing): https://aws.amazon.com/free/
- Install Trivy (container scanner): sudo apt install trivy (if available) or from GitHub releases
- Keep your Week 10 GitHub repository — Week 11 builds on it directly

Mini Project 3 Reminder

- Mini Project 3 is due at the end of Week 11
- You should have the vulnerable branch and working pipeline done this week
- Week 11 adds container scanning as a bonus — focus first on the core pipeline